



武汉大学
WUHAN UNIVERSITY

F7LY-OS

设计文档

队伍名称: F7LY
队伍成员: 曹子宸、郑喆宇、官恺祺
指导老师: 蔡朝晖

2025 年 8 月

目录

第一章 现场赛题目分析	1
1.1 基本加载	1
1.2 Git	1
1.2.1 文件系统	1
1.2.2 网络	2
1.3 Vim	2
1.4 GCC	3
第二章 技术路线	4
2.1 Git	4
2.1.1 内核支持	4
2.1.2 针对性优化	5
2.1.3 系统调用流程优化	5
2.1.4 ELF 文件与动态链接支持	6
2.2 GCC	6
2.2.1 概述	6
2.2.2 GCC 运行任务	6
2.2.3 技术实现	7
2.2.4 遇到的技术挑战	7
2.2.5 符号链接处理缺陷	8
2.2.6 实现成果	8
第三章 工程实践	10
3.1 问题及对应处理方法	10
3.1.1 运行时进程同步不一致	10
3.1.2 运行中内存不足	10
3.1.3 懒分配页面的权限问题	11
3.1.4 用户态传入的缺页异常	11
3.1.5 开发板程序行为与 QEMU 不一致	12
3.2 总结和未完成展望	12
3.2.1 已完成功能总结	12

3.2.2 现场赛未完成问题分析	13
3.2.3 整体改进策略	16

第一章 现场赛题目分析

本节对题目统筹分析，并结合自己的内核实现，分析现场赛题目中需要重点实现的需求和目前欠缺的点。

1.1 基本加载

现场赛中有四个软件：Git、gcc、rustc 和 vim。我们需要在内核中实现对它们的支持。而四个软件的通用处理即为加载 ELF 文件并执行。ELF 文件的加载主要分为以下几个步骤：

- 读取 ELF 文件头，检查文件格式是否正确。
- 读取程序头表，根据程序头表中的信息将各个段加载到内存中。
- 设置栈空间，准备好程序的运行环境。
- 跳转到程序的入口点，开始执行程序。

而这些文件在镜像中都是动态链接的，因此还需要加载动态链接器，并将动态链接器的路径传递给内核。需同时兼顾 QEMU 与物理板卡，页表与权限策略必须一致可复现。动态链接器加载后，通常内核中观测到的行为是动态链接器会先执行，运行期频繁 `mmap` / `munmap` / `mprotect` / `brk`，并伴随按需装载、权限切换与 COW。，然后再跳转到程序的入口点执行程序的代码，程序的装载会频繁访问这些库，对它们分配的内存需要正确处理权限以及大小。

1.2 Git

现场赛中 Git 有两方面功能，对操作系统的文件系统相关功能以及网络相关功能的要求较高。

1.2.1 文件系统

文件系统的主要功能是 `config`、`add`、`commit` 和 `log`，这些功能都需要对文件系统进行读写操作。

- **config**: 需要读取和写入配置文件，通常是一个文本文件。
- **add**: 需要将文件添加到暂存区，通常是将文件复制到 `.git` 目录下的 `index` 文件中。
- **commit**: 需要将暂存区的文件提交到版本库中，通常是将文件写入到 `.git` 目录下的 `objects` 目录中，并更新 `HEAD` 指针。

- **log**: 需要读取提交记录，通常是读取 `.git` 目录下的 `logs` 目录中的文件。

这些操作都需要对文件系统进行读写操作，因此需要完整文件系统的完备性。至少需要 `mkdir/rmdir`, `link/symlink/readlink`, `renameat2`（至少保证“原子重命名”语义），`getcwd` 等系统调用的支持。同时要求能够正确设置文件的权限和大小等元状态信息，并且能够正确处理符号链接等特殊文件：`stat/lstat/fstat` 这类系统调用的支持也需要正确处理。在这之中，`.git/index` 的锁文件约定（如 `index.lock`）是一种写入-同步-原子重命名，避免中断导致损坏。其在内核中的体现为 `open` 系统调用的各类标志位的支持，如 `O_EXCL`、`O_CREAT`、`O_TRUNC` 等。

1.2.2 网络

Git 的网络功能主要是从远程 `git repo` 执行 `clone` 操作，修改 `README`，把更新后的本地仓库上传/下载一个对应远程网站的一个自己的远程仓库中。这需要内核支持网络协议栈，能够正确处理 `TCP/IP` 协议，并且能够正确处理 `HTTP` 协议。

- **clone**: 需要从远程仓库下载代码，通常是通过 `HTTP` 协议进行下载。
- **push**: 需要将本地代码上传到远程仓库，通常是通过 `HTTP` 协议进行上传。
- **pull**: 需要从远程仓库下载代码，通常是通过 `HTTP` 协议进行下载。
- **remote**: 需要读取远程仓库的信息，通常是通过 `HTTP` 协议进行读取。

网络协议栈的基础框架需要支持 `TCP/IP` 协议，并且能够正确处理 `HTTP` 协议。需要能够正确处理网络连接的建立和断开，能够正确处理数据的发送和接收。

• 网络栈要点

- **TCP**: 三次握手、拥塞与重传的基本闭环，超时/半关闭处理；
- **DNS**: 如使用用户态解析器，确保 `getaddrinfo` 路径可用（文件或网络解析均可）；
- **定时器与可中断阻塞**: `connect` 超时、`recv` 超时与 `EAGAIN` 语义一致。

1.3 Vim

现场赛中 `Vim` 的主要功能是编辑文件，支持基本的文本编辑功能，包括

- 打开文件
- 保存文件
- 关闭文件

在正常使用 `vim` 时是从终端中打开的，需要内核支持终端设备的操作，能够正确处理终端设备的输入和输出。`Vim` 的编辑功能需要支持基本的文本编辑功能，包括插

入、删除、复制、粘贴等操作。但现场赛题的需求是创建一个 `hello.c` 文件，并在其中写入一行代码，因此只需要支持插入和保存操作即可。

1.4 GCC

现场赛中 GCC 的主要功能是编译 C 语言代码，支持基本的编译功能，包括

- 编译 C 语言代码
- 链接 C 语言代码
- 生成可执行文件

GCC 的编译功能需要支持基本的 C 语言语法和语义，包括变量声明、函数定义、控制结构等。现场赛题的需求是编译一个 `hello.c` 文件，并生成一个可执行文件，此处代码为 `printf("Hello World!\n");`，获取参数等编译步骤外，执行可执行文件、标准输出流的处理等都需要内核支持。GCC 的链接功能需要支持基本的链接功能，包括静态链接和动态链接。文件从 `.o` 文件链接到可执行文件，或从 `.so` 文件链接到可执行文件，需要支持 `gcc` 的链接器的运行。

第二章 技术路线

2.1 Git

本节详述 F7LY 操作系统对现场赛核心应用 Git 的技术支持路径。Git 作为分布式版本控制系统，其运行机制对操作系统的文件系统、内存管理和系统调用接口提出了严格要求。我们从程序行为分析入手，构建针对性的内核支持方案。

2.1.1 内核支持

2.1.1.1 目录扫描与文件状态检查

Git 的核心操作如 `git add` 和 `git status` 需要递归扫描工作目录，识别文件变更状态。这一过程的实现依赖于高效的目录遍历机制：

- **getdents64 系统调用：**Git 使用该系统调用批量获取目录项信息，避免逐一调用 `readdir` 带来的性能损耗。F7LY 内核实现了完整的 `getdents64` 支持，确保返回值格式严格符合 POSIX 标准，包括正确的 `d_reclen` 字段计算和 `d_type` 文件类型标识。

```
struct linux_dirent64 *d;  
struct linux_dirent64 *entry = (struct linux_dirent64 *);
```

- **文件状态查询优化：**Git 频繁使用 `faccessat`、`fstatat` 等系统调用检查文件访问权限和元数据。F7LY 针对这些调用优化了 VFS 层的属性缓存机制，减少底层文件系统的重复访问。

2.1.1.2 文件操作与 I/O 管理

Git 的文件操作模式对系统调用接口的正确性和性能提出了挑战：

- **原子文件操作：**`openat` 系统调用是 Git 文件操作的基础，支持相对路径操作和目录文件描述符传递。F7LY 实现了完整的 `openat` 族系统调用，确保路径解析的原子性和安全性。

2.1.1.3 内存映射与懒分配机制

Git 大量使用内存映射技术优化大文件处理和动态链接库加载：

- **文件映射支持：**Git 通过 `mmap` 将索引文件、包文件等映射到虚拟地址空间，实现零拷贝访问。F7LY 实现了完整的文件映射功能，支持 `MAP_PRIVATE`、`MAP_SHARED` 等映射模式，并通过页表机制实现写时拷贝（COW）。

```
// Git中典型的文件映射模式
fs::file *f = nullptr;
proc::Pcb *p = proc::k_pm.get_cur_pcb();
// 直接通过指针访问文件内容
```

- **懒分配策略：**针对 Git 的大文件映射需求，F7LY 实现了懒分配（lazy allocation）机制。仅在实际访问时分配物理页面，显著降低内存占用和启动时间。
- **动态链接库映射：**Git 依赖多个共享库，F7LY 在 `execve` 实现中正确解析 ELF 文件的动态段，建立准确的库文件映射，确保符号解析和重定位的正确性。

2.1.1.4 原子性操作保障

Git 使用锁文件机制保证并发安全，这对内核的原子操作支持提出了严格要求：

Git 通过创建 `.lock` 临时文件，写入完成后使用 `renameat2` 原子性重命名来更新 `index`、`config` 等关键文件。F7LY 实现了完整的 `renameat2` 系统调用，支持 `RENAME_NOREPLACE` 等标志位，确保操作的原子性。

```
// Git中的原子更新模式
int lock_fd = openat(dir_fd, "index.lock", O_CREAT | O_EXCL |
O_WRONLY, 0644);
// 写入新内容到lock文件
write(lock_fd, new_data, data_size);
fsync(lock_fd);
close(lock_fd);
// 原子性重命名替换原文件
renameat2(dir_fd, "index.lock", dir_fd, "index", RENAME_NOREPLACE);
```

2.1.2 针对性优化

2.1.3 系统调用流程优化

F7LY 针对 Git 的使用模式进行了深度优化：

- **getdents64 流程完善**: 通过分析 Git 的目录扫描模式, F7LY 优化了目录项缓存策略, 减少磁盘 I/O 次数。同时规范化了返回值处理, 确保与 glibc 的 `readdir` 实现完全兼容。
- **批量状态查询**: 针对 Git 频繁的文件状态检查, F7LY 实现了批量 `stat` 操作优化, 通过单次系统调用获取多个文件的元数据, 减少用户态与内核态切换开销。

2.1.4 ELF 文件与动态链接支持

Git 作为复杂的用户态程序, 其正确运行依赖于完善的动态链接支持:

- **ELF 解析规范化**: F7LY 在 `execve` 系统调用中实现了严格的 ELF 文件解析流程, 正确处理程序头表、节头表和动态段信息。确保各类段 (`.text`、`.data`、`.bss` 等) 的内存布局与权限设置符合 ABI 规范。
- **动态链接库装载**: 通过解析 ELF 文件的 `PT_INTERP` 段获取动态链接器路径, 正确建立链接器与目标程序的内存映射关系。同时处理 `PT_DYNAMIC` 段中的依赖库信息, 确保符号解析的正确性。
- **内存布局一致性**: 为保证 QEMU 环境与物理硬件的一致性, F7LY 统一了虚拟地址空间布局, 确保动态链接库的加载地址和权限设置在不同平台上保持一致。

这些技术措施的综合实施, 为 Git 在 F7LY 系统上的稳定运行提供了坚实的内核基础, 同时为其他复杂用户态程序的支持奠定了技术框架。

2.2 GCC

2.2.1 概述

GNU 编译器套件 (GNU Compiler Collection, GCC) 是一套由 GNU 项目开发的编程语言编译器套件, 支持多种编程语言, 包括 C、C++、Objective-C、Fortran、Ada 等。GCC 是自由软件, 以 GPL 许可证发布, 广泛应用于类 Unix 系统中。

在 F7LY-OS 中, 实现对 GCC 编译器的支持是验证系统完整性和兼容性的重要指标。GCC 的成功运行需要操作系统提供完整的进程管理、内存管理、文件系统和动态链接器支持。

2.2.2 GCC 运行任务

在操作系统竞赛中, GCC 编译器的运行任务主要包括以下两个核心测试用例:

2.2.2.1 基础功能测试

执行 `gcc --help` 命令，验证 GCC 编译器能够正常启动并输出帮助信息。该测试验证了：

- 程序加载与启动机制
- 基础的系统调用支持
- 动态链接库的正确加载

2.2.2.2 编译功能测试

使用 GCC 编译简单的 C 程序（如 `hello.c`），并链接标准库生成可执行文件。该测试验证了：

- 完整的编译工具链支持
- 文件系统的读写操作
- 进程创建与执行机制
- 复杂程序的内存管理

2.2.3 技术实现

2.2.3.1 动态链接支持

GCC 的运行涉及多个动态链接库的加载，包括但不限于：

- `ld-musl-riscv64.so.1`
- `libz.so.1`
- `LibLLVM-20.so`

F7LY-OS 通过完善的 ELF 加载器和动态链接器实现对这些库的正确加载和符号解析。

2.2.3.2 内存管理优化

为支持 GCC 等大型程序的运行，F7LY-OS 在内存管理方面进行了以下优化：

- 扩展虚拟内存空间（VMA）范围，提供充足的地址空间
- 实现懒分配机制，减少内存使用压力
- 优化 `mmap` 系统调用，支持大块内存映射

2.2.4 遇到的技术挑战

2.2.4.1 内存不足问题

问题描述: 在初期测试中, GCC 运行时频繁出现缺页异常, 导致程序无法正常执行。

根因分析: 经过详细调试发现, 问题源于虚拟内存地址空间(VMA)分配不足, 无法满足 GCC 及其依赖库的内存需求。

解决方案: 扩大 VMA 空间范围, 优化内存分配策略, 确保有足够的虚拟地址空间用于程序加载和运行。

2.2.5 符号链接处理缺陷

问题描述: GCC 编译过程中无法启动编译器前端 `cc1`, 导致编译流程中断。

根因分析: 问题出现在 `execve` 系统调用对符号链接的处理逻辑中, 当程序路径包含符号链接时, 路径解析出现错误。

解决方案: 修复 `execve` 系统调用中的符号链接解析逻辑, 确保能够正确处理符号链接路径, 使编译器能够正常启动子进程。

2.2.6 实现成果

F7LY-OS 在 GCC 支持方面取得了以下成果:

2.2.6.1 成功实现的功能

1. **GCC 基础功能:** 成功启动 GCC 编译器并执行 `gcc --help` 命令, 输出完整的帮助信息
2. **编译流程:** 完成 C 源文件的编译过程, 包括预处理、编译、汇编等阶段
3. **进程管理:** 正确处理 GCC 编译过程中的多进程创建与管理
4. **文件系统集成:** 支持源文件读取和目标文件写入操作

2.2.6.2 存在的限制

当前实现中, 链接阶段仍存在部分问题, 虽然完成的编译阶段, 但是链接出错使得无法生成可执行的 `elf` 文件:

- 动态链接器在处理复杂依赖关系时可能出现错误
- 部分系统调用的兼容性有待进一步完善

这些问题将在后续版本中持续优化和改进。

第三章 工程实践

由于现场赛当天发布题目，内核环境不可能直接支持运行，需要支持现场赛的四种硬件环境，所以在比赛期间免不了调试和准备工作。本节介绍我们在比赛中使用的调试方法。

3.1 问题及对应处理方法

3.1.1 运行时进程同步不一致

- 问题描述

现场赛中 `gcc` 等软件的运行包含的线程数量较多，常常出现线程调度不一致的问题，导致同样的操作在不同的运行中出现不同的结果，并且使用 `fork` 创建的子进程在退出后其一些子进程没有被正确托孤，导致僵尸进程的出现。主要体现在 `wait4` 系统调用等待的子进程并非预期的子进程，导致一些进程卡死无法往下走。

- 解决方法

- 我们仔细阅读了 `linux` 标准的 `wait4` 行为，发现其行为是等待任意子进程退出，而不是等待特定的子进程退出。因此我们修改了内核中的 `wait4` 实现，使其能够正确处理任意子进程的退出。
- 为了复刻真实的 `wait4` 行为，我们编译了一个 `git` 的运行脚本，在真实的 `linux` 环境中运行，观察其进程树和 `wait4` 调用的行为，并将其作为参考修改我们自己的内核，确保内核中的 `wait4` 实现与真实环境一致。

3.1.2 运行中内存不足

- 问题描述

现场赛中装载的动态链接库较多，且运行时频繁使用 `brk`, `mmap` 申请内存，导致内存不足的问题。主要体现在 `mmap` 系统调用返回 `ENOMEM` 错误，导致程序无法继续运行。

- 解决方法

- 经由调试日志打印，我们发现真正的问题在进程控制块中可存储的 `vma` 区域数量不足，导致无法为新的内存映射分配 `vma` 区域。而 `rustc` 装载的动态库较多，且每个动态库都需要一个 `vma` 区域，导致 `vma` 区域数量不足的问题。所以我们调

整了 vma 的存储结构，从静态数组改为动态创建，确保能够存储足够多的 vma 区域。

- 我们还增加了内存使用的调试日志，使用函数 `print_detailed_memory_usage` 打印内存使用情况，帮助我们更好地了解每一个进程的内存的使用情况，及时发现和解决内存不足的问题。

3.1.3 懒分配页面的权限问题

• 问题描述

现场赛中动态链接器和程序的运行时频繁使用 `mmap` 申请内存，并且这些内存区域通常是懒分配的，即在第一次访问时才真正分配物理页面并映射到虚拟地址空间中。而这些内存区域的权限通常是可读写的，不过动态链接库的代码权限通常是只读可执行的。因此在第一次访问这些懒分配页面时，可能会出现权限不匹配的问题，且分配可执行页面会带出更多的权限问题，导致程序无法继续运行。

• 解决方法

- 我们将页面分配逻辑根据缺页异常的类型不同进行区分，确保分配的页面权限与访问权限一致。对于可读写的页面，分配可读写的页面；对于只读可执行的页面，分配只读可执行的页面。这样可以确保在第一次访问懒分配页面时，权限匹配，避免权限不匹配的问题。

3.1.4 用户态传入的缺页异常

• 问题描述

用户态 `usertrap` 最常见的情况是缺页异常，这种问题经常在懒分配和写时拷贝逻辑中处理，但是一旦出现不包含在 vma 区域中的非法地址，就会导致内核发送 `SIGSEGV` 信号给用户态进程，进而导致进程崩溃。这种用户态的缺页地址总是无法预料的，且不便直接调试出现的原因。

• 解决方法

- 我们增加了内核中的调试日志，根据行为打印内存分配的详细记录，帮助我们更好地了解内存分配的过程，及时发现和解决问题。
- 更根本的解决办法是编译了带符号表的动态链接库和程序，使用 `gdb` 进行调试，观察缺页异常发生时的调用栈和内存状态，帮助我们更好地了解问题的根本原因，并进行针对性的修改。这样的行为较为困难，需要在 `gdb` 中直接查看寄存器和内存状态，才能从中找到出现空指针和野指针的原因。

3.1.5 开发板程序行为与 QEMU 不一致

- 问题描述

现场赛中需要同时支持 QEMU 和物理开发板两种环境，而开发板在运行 `cpp` 软件时，在程序对齐和页面权限的处理上与 QEMU 存在差异，导致同样的操作在两种环境中出现不同的结果，且开发板的调试手段较为有限，无法直接使用 `gdb` 进行调试。此处最主要的体现在于执行调度的 `swtch` 时，`Context` 结构体存储的位置破坏了其他全局变量，进一步导致进程分布的 `pid` 等信息被破坏。

- 解决方法

- 我们仔细审查了代码，打印了可疑变量的地址和大小，发现 `Context` 结构体存储的位置与其他全局变量存在冲突，导致变量被破坏。在初始化 `Context` 结构体后，我们发现未使用过的寄存器被错误地存储，导致结构体没有完全紧密排放，后面的全局变量存储的位置位于 `Context` 结构体的中间，导致变量被破坏。
- 在尝试多种解决方法后，我们最终选择在 `pcb` 的各种变量前面添加属性 `__attribute__((aligned(16)))`，确保这些变量按 16 字节对齐存储，避免与 `Context` 结构体冲突。这样可以确保变量不被破坏，程序在两种环境中都能正确运行。

3.2 总结和未完成展望

在本次操作系统内核设计和开发过程中，我们团队在系统调用实现、内存管理、文件系统等核心功能方面取得了显著进展。然而，在现场赛的四大应用场景测试中，仍存在一些关键技术问题未能完全解决，需要在后续工作中进一步改进和完善。

3.2.1 已完成功能总结

我们的 F7LY 操作系统在以下方面取得了重要突破：

1. **系统调用框架**：实现了完整的系统调用接口，支持文件操作、进程管理、内存管理等核心功能
2. **内存管理系统**：建立了虚拟内存管理机制，支持页面分配和回收
3. **文件系统**：实现了基本的文件系统操作，支持文件创建、读写、目录管理等
4. **进程管理**：支持进程创建、调度和同步机制
5. **设备驱动**：实现了基础的设备驱动框架

3.2.2 现场赛未完成问题分析

3.2.2.1 第一题：Git 功能实现问题

问题描述：Git 应用在网络相关功能（Task2）方面存在严重缺陷，无法完成远程仓库的克隆、推送和拉取操作。

具体表现：

- Task0（git -h）和 Task1（本地文件系统操作）基本可以运行
- Task2 中的网络操作（git clone、git push、git pull）完全失败
- 网络套接字相关系统调用支持不完善

根本原因：

1. 网络协议栈实现不完整，缺少 TCP/IP 协议的完整支持
2. SSL/TLS 协议支持缺失，无法处理 HTTPS 连接
3. DNS 解析功能未实现
4. 网络套接字系统调用（socket、connect、send、recv 等）功能不完善

改进路径：

优先级1：完善网络套接字系统调用

- 实现完整的socket()、bind()、listen()、accept()、connect()系统调用
- 路径：kernel/net/ 目录下补充socket.cc和tcp.cc实现

优先级2：集成轻量级TCP/IP协议栈

- 集成lwIP或类似轻量级网络协议栈
- 路径：thirdparty/lwip/ 目录下集成并配置

优先级3：DNS解析支持

- 实现基础DNS客户端功能
- 路径：kernel/net/dns.cc

优先级4：SSL/TLS支持（可选）

- 集成mbedtls或OpenSSL轻量版本
- 路径：thirdparty/ssl/ 目录下集成

3.2.2.2 第二题：Vim 功能实现问题

问题描述: Vim 编辑器在终端显示和交互编辑方面存在严重问题,无法正常进行文本编辑操作。

具体表现:

- `vim -h` 可以显示帮助信息
- `vim hello.c` 启动后界面显示异常,无法正常编辑
- 终端控制字符处理不正确,光标移动和屏幕刷新有问题

根本原因:

1. 终端模拟器功能不完善,缺少完整的 ANSI 转义序列支持
2. `ioctl` 系统调用对终端控制的支持不足
3. 信号处理机制不完善,影响终端中断处理
4. 字符设备驱动的非阻塞 I/O 支持不完整

改进路径:

优先级1: 完善终端设备驱动

- 实现完整的ANSI转义序列解析
- 路径: `kernel/devs/terminal.cc` 中补充转义序列处理

优先级2: 增强`ioctl`系统调用

- 添加TCGETS、TCSETS等终端控制命令支持
- 路径: `kernel/sys/ioctl.cc` 中补充终端相关命令

优先级3: 完善信号系统

- 实现SIGINT、SIGTERM等信号的正确处理
- 路径: `kernel/proc/signal.cc` 中补充信号处理逻辑

优先级4: 非阻塞I/O支持

- 实现`select()`、`poll()`等I/O多路复用机制
- 路径: `kernel/sys/select.cc` 新增实现

3.2.2.3 第三题: GCC 功能实现问题

问题描述: GCC 编译器在链接阶段存在严重问题,无法正确生成可执行文件。

具体表现:

- `gcc -h` 正常显示帮助
- `gcc hello.c` 编译过程可以进行,但链接器调用失败

- 缺少系统标准库的正确链接

根本原因：

1. 动态链接器支持不完善，ld.so 功能缺失
2. 系统调用与 C 库的接口映射不正确
3. ELF 文件加载器对动态链接的支持不足
4. 缺少完整的 C 运行时库（crt0.o 等）

改进路径：

优先级1：实现动态链接器

- 开发基础的动态链接器ld.so
- 路径：user/deps/ld/ 目录下实现动态链接器

优先级2：完善ELF加载器

- 支持动态链接的ELF文件加载
- 路径：kernel/fs/elf.cc 中补充动态链接支持

优先级3：C运行时库补充

- 提供完整的crt0.o、crti.o、crtn.o
- 路径：user/deps/crt/ 目录下实现运行时对象

优先级4：系统调用接口优化

- 确保libc与内核系统调用接口的完全兼容
- 路径：kernel/sys/ 和 user/syscall_lib/ 同步优化

3.2.2.4 第四题：Rustc 功能实现问题

问题描述：Rust 编译器在初始化时的内存分配出现严重问题，导致编译过程无法正常进行。

具体表现：

- rustc -h 启动时即出现内存分配错误
- 编译过程中出现段错误或内存访问异常
- Rust 运行时的内存管理与系统不兼容

根本原因：

1. 内存分配器实现存在 bug，特别是大内存块分配
2. 堆内存管理的并发安全性不足

3. 虚拟内存映射机制与 Rust 运行时不兼容
4. 系统调用 `mmap()`、`munmap()` 实现不完善

改进路径:

优先级1: 修复内存分配器

- 重新设计堆内存分配算法, 支持大块内存分配
- 路径: `kernel/mem/heap.cc` 重构分配算法

优先级2: 完善内存映射系统调用

- 实现完整的 `mmap()`、`munmap()`、`mprotect()` 系统调用
- 路径: `kernel/mem/mmap.cc` 补充完整实现

优先级3: 增强并发内存管理

- 添加内存分配的锁机制和线程安全保护
- 路径: `kernel/mem/allocator.cc` 添加并发控制

优先级4: Rust 运行时适配

- 调研 Rust 内存模型, 确保系统兼容性
- 路径: 研究 Rust runtime 与内核接口适配方案

3.2.3 整体改进策略

基于以上分析, 我们建议按照以下策略进行系统完善:

短期目标 (1-2 个月):

1. 优先解决内存管理问题, 确保大型应用的基础运行环境
2. 完善终端设备驱动, 支持基本的文本编辑功能
3. 建立基础的网络协议栈框架

中期目标 (3-6 个月):

1. 实现完整的动态链接器系统
2. 集成成熟的 TCP/IP 协议栈
3. 完善信号处理和 I/O 多路复用机制

长期目标 (6 个月以上):

1. 支持 SSL/TLS 和完整的网络安全协议
2. 建立完善的开发工具链生态
3. 实现与主流编程语言运行时的完全兼容

通过系统性的改进，F7LY 操作系统将能够支持现代软件开发的完整工具链，为用户提供完整的操作系统体验。